# Weather Streaming Project – 10 villes françaises

## 1- Objectif du projet

Ce projet vise à collecter, traiter et visualiser en temps réel les données météo de 10 villes françaises :

**Paris, Lyon, Marseille, Toulouse, Bordeaux, Lille, Nice, Nantes, Strasbourg, Montpellier**.

L'objectif est de créer un pipeline complet :

1. **Extraction automatique** des données météo via l'API OpenWeatherMap.
2. **Transmission en temps réel** des données via Kafka.
3. **Traitement et transformation** avec Spark Structured Streaming.
4. **Stockage centralisé** dans un Data Lake en fichiers Parquet.
5. **Visualisation interactive** via Streamlit avec KPI et graphiques enrichis.

---

## 2- Architecture du projet

**API Producer (src/api_producer.py)**

```
│ - Récupère les données météo toutes les 10 secondes
│ - Envoie à Kafka sur le topic 'raw_api_events'
▼
```

**Kafka Topic (raw_api_events)**

```
│ - Permet le flux en temps réel
▼
```

**Spark Structured Streaming (src/spark_streaming.py)**

```
│ - Lit les données depuis Kafka
│ - Applique les transformations :
```

│ • Création colonne `alert_temp` (High temp si >30°C)

│ • Création colonne `temp_diff` (max - avg temp, pour l'instant = 0)

│ - Stocke les fichiers transformés dans Data Lake Parquet

▼

**Data Lake Parquet (data/weather_lake/)**

│ - Contient toutes les observations horodatées

▼

**Streamlit Dashboard (src/dashboard.py)**

│ - Dashboard interactif et attractif

│ - KPI : température moyenne, humidité moyenne, ville la plus chaude

│ - Graphiques :

│ • Température moyenne par ville (barres)

│ • Evolution température par ville (ligne)

│ • Humidité moyenne par ville (diagramme circulaire)

│ • Top 5 villes les plus chaudes, froides et humides

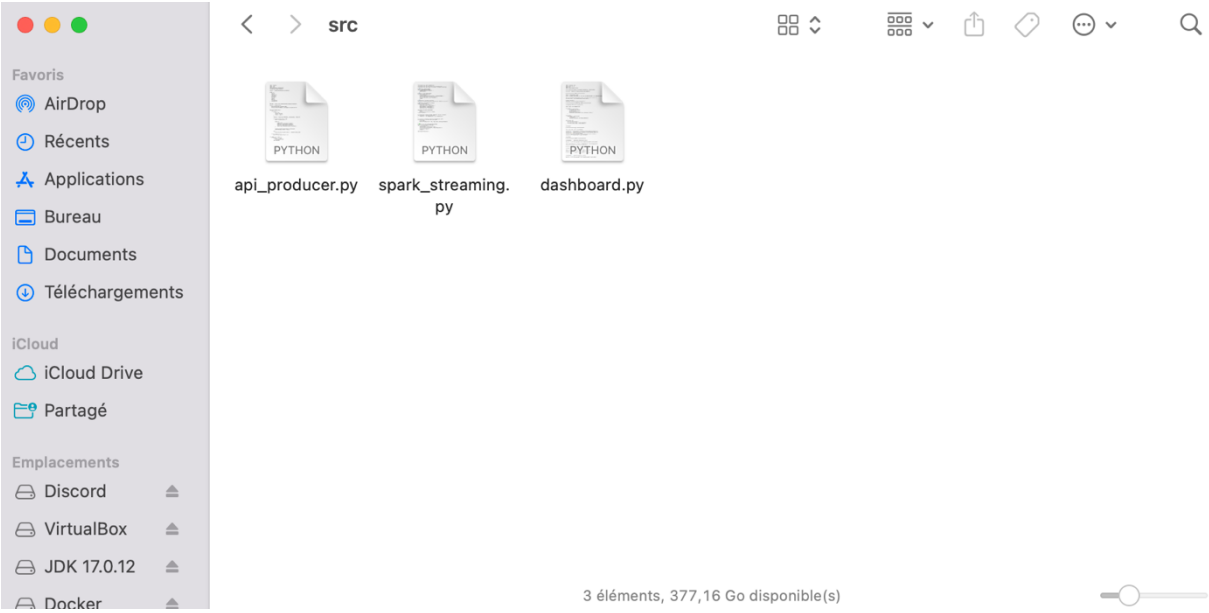
│ - Table des dernières observations

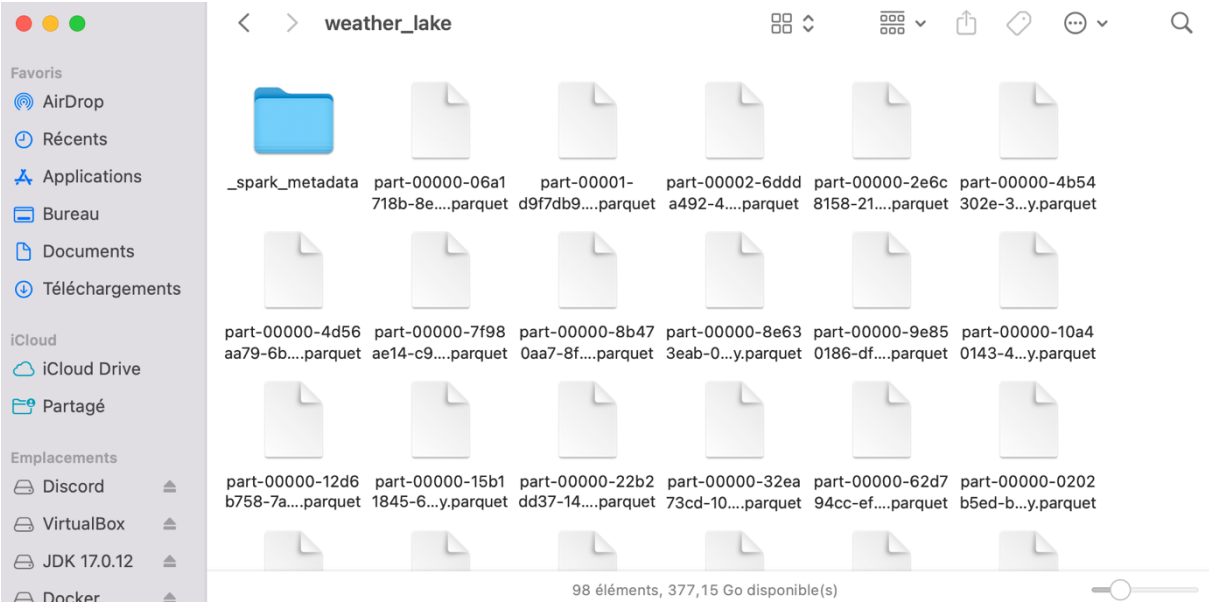
---

## 3- Structure du projet

```
realtime-spark/
│
├─ src/
│   ├─ api_producer.py       # Récupération automatique des données météo et envoi à Kafka
│   ├─ spark_streaming.py    # Traitement avec Spark et stockage Parquet
│   └─ dashboard.py          # Dashboard Streamlit interactif
```

```
│
├── data/
│   └── weather_lake/          # Data Lake (fichiers Parquet)
│
├── README.md                  # Ce fichier
└── requirements.txt           # Librairies Python nécessaires
```



## 4- Colonnes et transformations

Les fichiers Parquet contiennent les colonnes suivantes :

| Colonne | Description |
|---|---|
| city | Nom de la ville |
| temperature | Température actuelle en °C |
| humidity | Humidité en % |
| pressure | Pression atmosphérique en hPa |
| event_time | Horodatage de l'observation |
| temp_diff | Différence max-temp / moyenne (actuellement = 0) |
| alert_temp | "High temp" si température >30°C, "Normal" sinon |

**Remarques** :

- La colonne temp_diff n'est pas affichée dans le dashboard car elle contient uniquement 0 pour l'instant.
- alert_temp est utilisée pour mettre en évidence les températures élevées dans les graphiques et KPI.

---

## 5- Installation et exécution

### 5.1 Créer un environnement virtuel et installer les dépendances

```
python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt
```

### 5.2 Lancer Kafka

Assurez-vous que Kafka et Zookeeper tournent sur localhost:9092.
Exemple :

brew services start kafka

brew services start zookeeper

### 5.3 Lancer le Producer pour récupérer les données

python3 src/api_producer.py

### 5.4 Lancer Spark Streaming pour traiter et stocker les données

spark-submit src/spark_streaming.py

### 5.5 Lancer le dashboard Streamlit

streamlit run src/dashboard.py

---

## 6- Dashboard Streamlit – fonctionnalités

1. **Filtres interactifs** :
    - Choix des villes (sélection multiple).
    - Période personnalisée avec icône calendrier.
2. **KPI temps réel** :
    - Température moyenne
    - Humidité moyenne
    - Ville la plus chaude
    - Nombre d'observations
3. **Graphiques** :
    - Température moyenne par ville (barres).
    - Evolution des températures par ville (ligne).

    - Humidité moyenne par ville (diagramme circulaire).
4. **Top 5 villes** :
    - Les plus chaudes
    - Les plus froides
    - Les plus humides

5. **Table de données récentes** :
    - Dernières 50 observations, sans colonne temp_diff.

**Lien GitHub :** https://github.com/abidhiaf/realtime-weather-streaming.git

**Remarque :** Le projet a été développé et testé dans un environnement **macOS**. Certaines commandes ou chemins peuvent différer sous Linux.

- Restons à votre disposition pour tout complément d'information.

**Ahmed ABIDHIAF**